

 MONGODB ATLAS SEARCH

BM25 Index Creation & Lucene Analyzer Explained

A deep dive into how MongoDB Atlas Search uses Apache Lucene under the hood — covering tokenization, BM25 scoring, analyzer selection, and real-world search patterns.



Apache Lucene

Core search library powering indexing and queries

Text Indexing

Tokenization, analyzers, and inverted index creation

BM25 Scoring

Relevance ranking using Lucene's BM25 algorithm

MongoDB Atlas Search → Built on Lucene

MongoDB Atlas Search is not a standalone search engine — it is built **on top of Apache Lucene**, one of the most powerful open-source search libraries in the world. This is why you encounter Lucene-specific analyzer names in your index definitions.

What Lucene Powers

- Tokenization of raw text
- Keyword search and exact matching
- BM25 relevance scoring
- Full-text indexing
- Document ranking

Lucene Analyzers You'll See

- `lucene.standard` — for full-text search
- `lucene.keyword` — for exact matching

These analyzers are the core decision-makers for how your data is stored and retrieved in the index.

Why the Analyzer Matters

The analyzer is the gatekeeper of your search index. It determines everything about how text is processed — from ingestion to retrieval.



How Text is Tokenized

The analyzer decides whether a sentence like *"Login functionality failed"* is broken into individual words or stored as a single value.



How Words are Stored

Tokens are stored in the index. The analyzer controls whether words are lowercased, punctuation is removed, and stop words are filtered.



How Queries are Matched

At query time, the same analyzer processes the search input — ensuring the query tokens align with the stored index tokens for accurate retrieval.



Without an analyzer, the search engine cannot split words, ignore case, remove punctuation, or support full-text search. Choosing the wrong analyzer leads to inaccurate or broken search results.

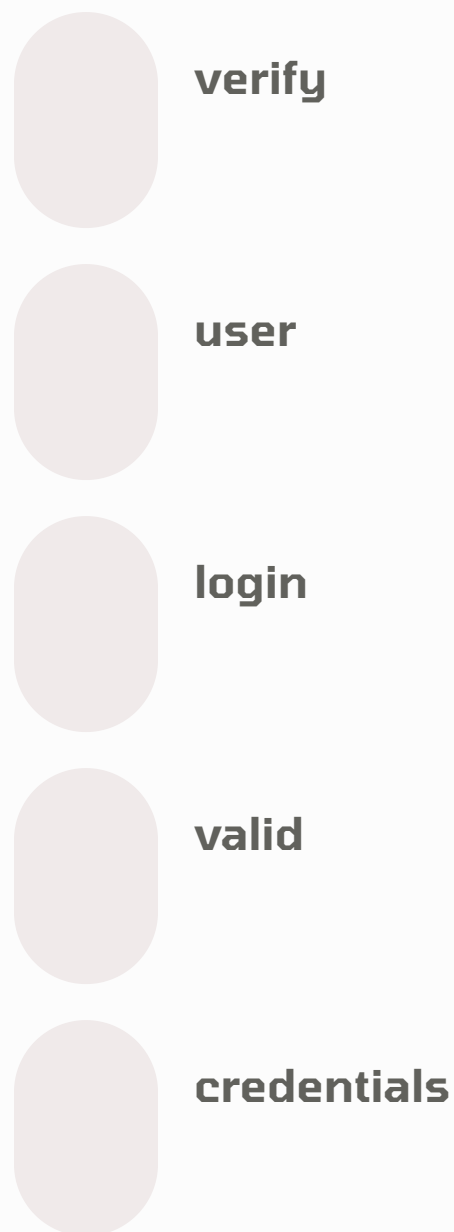
The Standard Analyzer — Full-Text Search

lucene.standard is designed for **human-readable text fields**. It breaks sentences into individual tokens, enabling BM25 relevance ranking and partial keyword matching.

How It Works

Given the title: *"Verify user can login with valid credentials"*

lucene.standard converts it into tokens:



Now if a user searches **"login validation"**, BM25 can still retrieve this test case by matching relevant tokens.

Best Used For

- Search relevance and BM25 scoring
- Partial and semantic keyword matching
- Text retrieval from natural language

Example Fields

- title
- description
- steps
- expectedResults
- module

The Keyword Analyzer — Exact Matching

`lucene.keyword` treats the **entire field value as a single token**. No splitting, no lowercasing, no transformation. It is designed for structured metadata fields where you need precise, exact filtering.

The Problem with Standard on IDs

If `id = "TC_LOGIN_001"` were indexed with `lucene.standard`, it would be tokenized into:

- `tc`
- `login`
- `001`

Search for `TC_LOGIN_001` would become inaccurate and unreliable.

With `lucene.keyword`

`id = "TC_LOGIN_001"` is stored exactly as:

| `"TC_LOGIN_001"`

Searching `TC_LOGIN_001` → **Exact match found**. No ambiguity, no false positives.

Fields That Use This

- `id`, `priority`, `type`
- `version`, `createdBy`
- `automationManual`

How BM25 Scoring Works

BM25 (Best Match 25) is a probabilistic ranking algorithm. It calculates a relevance score for each document based on the query terms. It **only works on tokenized fields** — which is why `lucene.standard` is essential for BM25-powered search.

1

Term Frequency

How many times does the search word appear in the document?
More occurrences = higher relevance signal.

2

Inverse Document Frequency

How rare is the word across all documents? Rare words carry more weight than common ones like "the" or "is".

3

Relevance Score

BM25 combines both signals to produce a final score. Documents are ranked from highest to lowest score.

Example: Search query = *"payment failure"*

Test Case A: *"Verify payment failure handling"* → BM25 Score: **HIGH**

Test Case B: *"Check logout"* → BM25 Score: **LOW**

Because "payment" and "failure" both appear in Test Case A, it ranks significantly higher.

Why dynamic: false?

Setting `dynamic: false` in your index definition means **only the fields you explicitly specify will be indexed**. MongoDB Atlas Search will not automatically index new or unknown fields that appear in your documents.

✓ Benefits

- Better query performance
- Smaller index size on disk
- Controlled, predictable search behavior
- Avoids unwanted field indexing

Practical Example

Suppose a document suddenly contains a random field:

```
tempData: "xyz"
```

With `dynamic: false`, this field is **completely ignored** by the index. It won't consume index space, won't appear in search results, and won't interfere with your carefully designed search behavior.

- ❗ This is especially important in production systems where document schemas may evolve over time or contain debug/temporary fields.

Choosing the Right Analyzer Per Field

The decision of which analyzer to use comes down to one question: **Is this field human-readable text, or structured metadata?**

Field	Analyzer	Purpose	Search Type
title	lucene.standard	Human-readable text	BM25 relevance
description	lucene.standard	Natural language	Full-text search
steps	lucene.standard	Sentence content	Partial matching
expectedResults	lucene.standard	Descriptive text	Keyword retrieval
id	lucene.keyword	Unique identifier	Exact lookup
priority	lucene.keyword	Structured metadata	Exact filter
type	lucene.keyword	Category value	Exact filter
version	lucene.keyword	Version string	Exact filter

The Two Analyzers — A Simple Mental Model

If you ever forget which analyzer to use, remember this analogy:

lucene.standard = Word Shredder

Takes a sentence and breaks it into individual searchable words.

"Payment failed during checkout"

→ payment · failed · during · checkout

Each word becomes a searchable token. BM25 can now score and rank documents based on these tokens.

lucene.keyword = Sealed Envelope

Takes a value and stores it completely untouched.

"HIGH_PRIORITY"

→ HIGH_PRIORITY (stored exactly as-is)

No splitting, no transformation. You must search the exact value to get a match. Perfect for IDs, enums, and categories.

- ✓ The golden rule: If a human wrote it in a sentence → **lucene.standard**. If a system generated it as a code or category → **lucene.keyword**.

The Combined Search Pattern in Action

The most powerful search systems combine **both analyzers together** — using BM25 for relevance on text fields and exact filtering on metadata fields. This pattern is extremely common across modern search architectures.



Example Query Concept

Search: *"payment failure"*

Filter: priority = High · type = Smoke

- BM25 handles relevance scoring on title, description, steps
- Keyword fields handle exact filtering on priority and type

Where This Pattern Appears

- RAG (Retrieval-Augmented Generation) systems
- Jira and issue tracker search
- Test case retrieval platforms
- E-commerce product search
- Enterprise knowledge bases
- Document management systems